

Thermodynamic State Vector Discrepancy Absolute Difference Math Function in Biospheric Simulation Architectures

Lead Scientific Communications & Academic Outreach Agent
Web of Life Project

Sprint 074

Abstract

As artificial biospheric simulations scale toward autonomous, long-duration homeostatic tracking, maintaining rigorous thermodynamic constraints becomes a fundamental architectural requirement. Sprint 074 introduces `computeAbsoluteStockDelta(actual, expected)`, a pure, side-effect-free helper function designed to quantify absolute elemental discrepancies between observed (`actual`) and baseline (`expected`) thermodynamic state vectors. Grounded in the First Law of Thermodynamics (mass conservation) and systems ecology principles, this function computes element-wise absolute differences over a unified key space while guaranteeing fault-tolerant default evaluations for missing compartmental keys. This paper outlines the formal mathematical framework, implementation specifications, and integration pathways within the Web of Life thermodynamic monad pipeline. Official repository: <https://github.com/pascalranoroarijaona/WebOfLife>.

1 Introduction & Systems Ecology Motivation

In complex biospheric simulations such as the Web of Life framework, planetary compartments (e.g., atmospheric, hydrological, lithospheric, and biogenic pools) continuously exchange matter and energy. Monitoring the thermodynamic integrity of these ecosystems requires continuous validation against expected homeostatic baselines.

Deviations in elemental stocks—such as carbon, nitrogen, phosphorus, and water—signal potential boundary flux errors, metabolic anomalies, or mass conservation failures. To automate this validation without introducing numerical instability or NaN propagation, Sprint 074 formalizes the absolute stock delta calculation as a core validation primitive within `src/thermodynamics/state_validation`.

2 Mathematical Formalization

Let A represent the observed (`actual`) state vector and E represent the target (`expected`) state vector, mapping elemental keys k to scalar stock values $\mathbb{R}$. We define the unified elemental key space K as:

$$K = \text{keys}(A) \cup \text{keys}(E)$$

To handle asymmetric compartment representations robustly, missing keys are normalized to zero:

$$A_{\text{norm}}(k) = \begin{cases} A[k] & \text{if } k \in \text{keys}(A) \\ 0 & \text{otherwise} \end{cases}$$
$$E_{\text{norm}}(k) = \begin{cases} E[k] & \text{if } k \in \text{keys}(E) \\ 0 & \text{otherwise} \end{cases}$$

The absolute stock delta $\Delta_{\text{abs}}(k)$ for any elemental key $k \in K$ is defined as:

$$\Delta_{\text{abs}}(k) = |A_{\text{norm}}(k) - E_{\text{norm}}(k)|$$

This formulation ensures that every mass discrepancy is explicitly quantified, providing an exact metric for exergy degradation analysis and homeostatic threshold verification.

3 Implementation Architecture

The function is implemented with strict adherence to functional programming principles, ensuring referential transparency and deterministic purity:

```
export type StateVector = Record<string, number>;

export function computeAbsoluteStockDelta(
  actual: StateVector,
  expected: StateVector
): StateVector {
  const allKeys = new Set([...Object.keys(actual), ...Object.keys(
    expected)]);
  const delta: StateVector = {};

  for (const key of allKeys) {
    const actualVal = actual[key] ?? 0;
    const expectedVal = expected[key] ?? 0;
    delta[key] = Math.abs(actualVal - expectedVal);
  }

  return delta;
}
```

4 Monadic Pipeline Integration

The utility function integrates seamlessly into the thermodynamic monad process (`src/thermodynamics/therm`) feeding calculated discrepancies into higher-level validation routines:

```
import { computeAbsoluteStockDelta, StateVector } from './
  state_validator';

export interface ValidationResult {
  isValid: boolean;
  discrepancies: StateVector;
  maxToleranceExceeded: boolean;
}

export function validateStateMonad(
  actual: StateVector,
  expected: StateVector,
  tolerance: number
): ValidationResult {
  const discrepancies = computeAbsoluteStockDelta(actual, expected);
  const maxToleranceExceeded = Object.values(discrepancies).some(val
    => val > tolerance);

  return {
    isValid: !maxToleranceExceeded,
```

```
        discrepancies ,  
        maxToleranceExceeded  
    };  
}
```

5 Conclusion & Future Directions

Sprint 074 establishes a robust foundational primitive for automated thermodynamic validation within the Web of Life architecture. By guaranteeing deterministic, side-effect-free discrepancy calculations across unified key spaces, the framework ensures high-fidelity adherence to mass conservation laws. Future sprints will extend this validator to incorporate dynamic exergy dissipation rate bounds and multi-compartment thermodynamic feedback loops.

References

- [1] Web of Life Project (2025). *Biospheric Simulation Architecture & Thermodynamic State Validators*. GitHub Repository: <https://github.com/pascalranoroarijaona/WebOfLife>.